

using `std::optional` and `std::move` to model cycle-approximate pipeline stalls provides a highly readable and elegant framework for cycle-approximate architectural modeling

In a cycle-approximate pipeline loop (typically driven by an `SC_METHOD` or `SC_THREAD` clocked sensitive block), a stage might contain a valid hardware instruction/packet, or it might be stalled (bubbled). Instead of tracking validity via manual, error-prone companion booleans (`bool stage1_valid`), wrapping the processing transaction inside a `std::optional` encapsulates both the state of data validity and the data itself into a single atomic variable.

Here is a complete SystemC implementation of a 3-stage instruction pipeline (Fetch, Decode, Execute) utilizing `std::optional` to manage pipeline bubbles and structural stalls dynamically.

1. Structural Pipeline Components

We will define a basic `Instruction` struct and create a pipeline module that updates its internal stages every clock cycle.

```
#include <systemc.h>
#include <optional>
#include <queue>
#include <iostream>
#include <iomanip>

// 1. Definition of the processing element
struct Instruction {
    uint32_t pc;
    std::string op;
    uint32_t delay_cycles; // Simulates structural stall dependency (e.g., multiplier latency)
};

SC_MODULE(PipelineModel) {
    sc_in<bool> clk;

    // Pipeline stages modeled via std::optional.
    // std::nullopt explicitly represents a pipeline "bubble" or "stall".
```

```

std::optional<Instruction> fetch_stage = std::nullopt;
std::optional<Instruction> decode_stage = std::nullopt;
std::optional<Instruction> execute_stage = std::nullopt;

// Architectural input stimulus queue
std::queue<Instruction> instruction_mem;

SC_CTOR(PipelineModel) {
    SC_METHOD(tick);
    sensitive << clk.pos();
    dont_initialize();

    // Seed mock instructions into our memory stream
    instruction_mem.push({0x1000, "ADD", 0});
    instruction_mem.push({0x1004, "MUL", 2}); // This instruction forces a 2-cycle stall!
    instruction_mem.push({0x1008, "SUB", 0});
    instruction_mem.push({0x100C, "AND", 0});
}

void tick() {
    std::cout << "\n--- [Cycle " << sc_time_stamp() << "]" << "\n";

    //-----
    // STAGE 3: EXECUTE
    //-----
    // .has_value() evaluates whether the pipeline stage contains an instruction
    if (execute_stage.has_value()) {
        if (execute_stage->delay_cycles > 0) {
            // Structural Stall Condition: The functional unit is busy!
            std::cout << "[EXECUTE] STALL: " << execute_stage->op
                << " busy. Remaining: " << execute_stage->delay_cycles << " cycles.\n";

            // Decrement the remaining busy execution cycles
            execute_stage->delay_cycles--;
        }
    }
}

```

```

        // Block previous stages from advancing by returning immediately
        print_pipeline_state();
        return;
    } else {
        // Clean Execution completion
        std::cout << "[EXECUTE] Retired Instruction: " << execute_stage->op
            << " (PC: 0x" << std::hex << execute_stage->pc << std::dec << ")\n";

        // Clear the stage entirely, converting it back to std::nullopt (bubble)
        execute_stage = std::nullopt;
    }
} else {
    std::cout << "[EXECUTE] Bubble (Empty)\n";
}

//-----
// STAGE 2: DECODE
//-----
if (decode_stage.has_value()) {
    std::cout << "[DECODE] Processing: " << decode_stage->op << "\n";

    // Move structural ownership downstream into the execution phase
    execute_stage = std::move(decode_stage);

    // Explicitly clear the source stage to turn it back into a bubble
    decode_stage = std::nullopt;
} else {
    std::cout << "[DECODE] Bubble (Empty)\n";
}

//-----
// STAGE 1: FETCH
//-----
if (fetch_stage.has_value()) {
    std::cout << "[FETCH] Passing to Decode: " << fetch_stage->op << "\n";

```

```

        decode_stage = std::move(fetch_stage);
        fetch_stage = std::nullopt;
    }

    // Pull a fresh instruction from memory if available
    if (!instruction_mem.empty()) {
        fetch_stage = instruction_mem.front();
        instruction_mem.pop();
        std::cout << "[FETCH]  Fetched New: " << fetch_stage->op << "\n";
    } else {
        fetch_stage = std::nullopt;
    }

    print_pipeline_state();
}

// Helper debugging matrix output
void print_pipeline_state() {
    auto format_stage = [](const std::optional<Instruction>& stage) {
        return stage.has_value() ? stage->op : "---";
    };
    std::cout << "  [Pipeline Status] F: " << format_stage(fetch_stage)
        << " | D: " << format_stage(decode_stage)
        << " | E: " << format_stage(execute_stage) << "\n";
}
};

```

2. Main Simulation Harness

Binding a mock clock loop to execute the pipeline architecture trace:

```

int sc_main(int argc, char* argv[]) {
    // Generate an arbitrary 10ns simulation clock line
    sc_clock clk("clk", 10, SC_NS);
}

```

```

PipelineModel pipeline("pipeline_FetchDecodeExecute");
pipeline.clk(clk);

sc_start(80, SC_NS); // Step out across 8 cycles
return 0;
}

```

How the `std::optional` Stall Logic Behaves in Execution

When executing this simulation, the `MUL` instruction hits the execute stage and triggers the structural stall branch. Because the loop returns early, the `std::move` commands for lower stages are skipped, perfectly preserving the state of upstream stages without needing explicit stall logic wires:

```

--- [Cycle 10 ns] ---
[EXECUTE] Bubble (Empty)
[DECODE]  Bubble (Empty)
[FETCH]   Fetched New: ADD
[Pipeline Status] F: ADD | D: --- | E: ---

--- [Cycle 20 ns] ---
[EXECUTE] Bubble (Empty)
[DECODE]  Bubble (Empty)
[FETCH]   Passing to Decode: ADD
[FETCH]   Fetched New: MUL
[Pipeline Status] F: MUL | D: ADD | E: ---

--- [Cycle 30 ns] ---
[EXECUTE] Bubble (Empty)
[DECODE]  Processing: ADD
[FETCH]   Passing to Decode: MUL
[FETCH]   Fetched New: SUB
[Pipeline Status] F: SUB | D: MUL | E: ADD

--- [Cycle 40 ns] ---

```

```

[EXECUTE] Retired Instruction: ADD (PC: 0x1000)
[DECODE] Processing: MUL
[FETCH] Passing to Decode: SUB
[FETCH] Fetched New: AND
[Pipeline Status] F: AND | D: SUB | E: MUL

--- [Cycle 50 ns] ---
[EXECUTE] STALL: MUL busy. Remaining: 2 cycles.
[Pipeline Status] F: AND | D: SUB | E: MUL

--- [Cycle 60 ns] ---
[EXECUTE] STALL: MUL busy. Remaining: 1 cycles.
[Pipeline Status] F: AND | D: SUB | E: MUL

```

TO-DO .. option to allow ..

i) **bubbles collapsing**, or

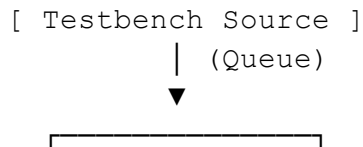
ii) something like the **decode stage also supporting multi-cycle instructions** (i.e. the instruction struct will need to allow individual delta_cycle for each stage) whose execution should NOT be stopped/prevented by another multi-cycle instruction executing in execute stage, or even each pipe stage i (0 to 2 in the case here) taking corresponding Ci cycles based on its primary input dynamically, goal is to have a modular re-usable component for a lot of use-cases

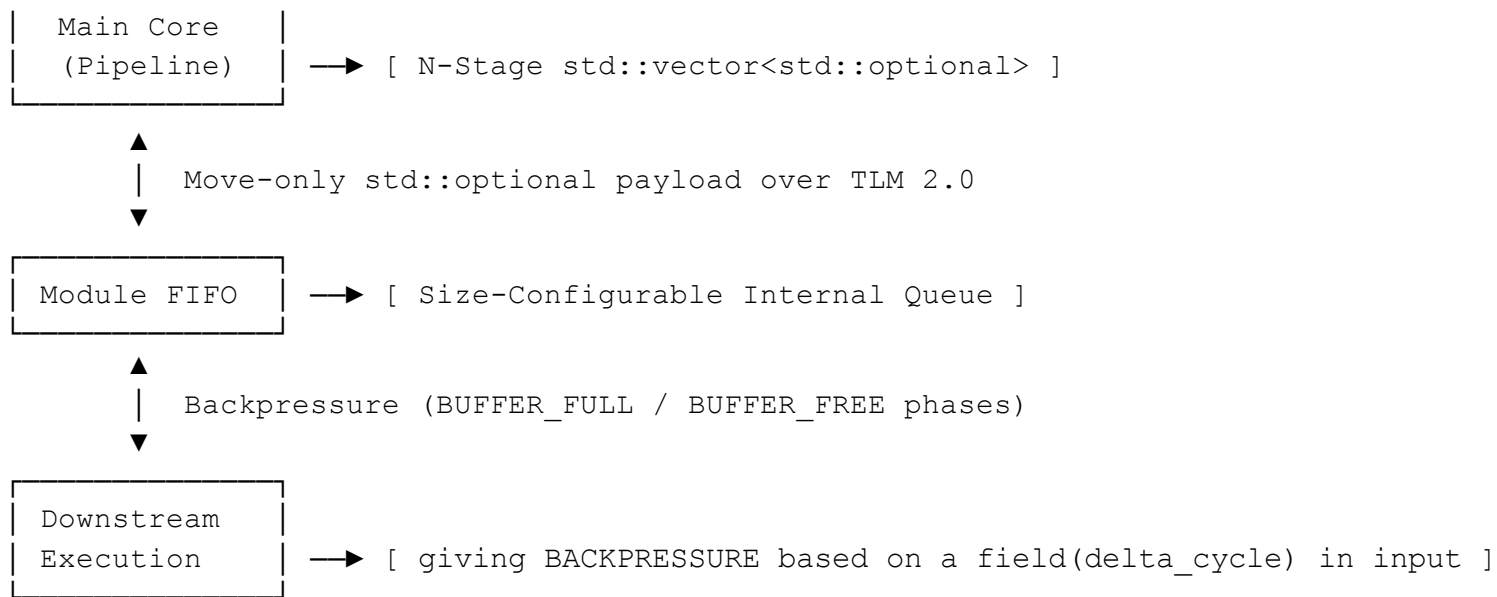
Done (see next example): back-pressure from downstream **and** having both number of pipe stages / FIFO size programmable ..

i)

multi-sc_module example with back-pressure from downstream, both number of pipe stages and FIFO size programmable, while using

tlm_utils::simple_initiator_socket, tlm_utils::simple_target_socket, nb_transport_fw, nb_transport_bw, phase extension, customized payload type, along with ModernC++ like std::optional, std::nullopt and std::move





```
#include <systemc>
#include <tlm>
#include <tlm_utils/simple_initiator_socket.h>
#include <tlm_utils/simple_target_socket.h>
#include <optional>
#include <queue>
#include <vector>
#include <string>
#include <iostream>
#include <iomanip>

namespace custom_phase {
    DECLARE_EXTENDED_PHASE(BUFFER_FULL);
    DECLARE_EXTENDED_PHASE(BUFFER_FREE);
}

struct Instruction {
    uint32_t pc;
```

```

    std::string op;
    uint32_t delay_cycles = 0; // used differently as compared to an earlier example
};

struct CA_Traits { // as compared to: tlm::tlm_base_protocol_types
    using tlm_payload_type = std::optional<Instruction>; // vs: tlm::tlm_generic_payload
    using tlm_phase_type    = tlm::tlm_phase;
};

// =====
// MODULE 1: MAIN CORE PIPELINE - N-STAGE PROGRAMMABLE
// =====
class ProgrammablePipeline : public sc_core::sc_module {
public:
    tlm_utils::simple_initiator_socket<ProgrammablePipeline, 32, CA_Traits> init_socket;
    // tlm_utils::simple_initiator_socket<MODULE, BUSWIDTH, TYPES>
    // 32 -> could have used: #define/const/constexpr
    sc_core::sc_in<bool> clk;

    SC_HAS_PROCESS(ProgrammablePipeline);

    ProgrammablePipeline(sc_core::sc_module_name name, size_t num_stages)
        : sc_core::sc_module(name), init_socket("init_socket"), m_num_stages(num_stages)
    {
        init_socket.register_nb_transport_bw(this, &ProgrammablePipeline::nb_transport_bw);
        m_stages.resize(m_num_stages, std::nullopt);

        SC_METHOD(tick);
        sensitive << clk.pos();
        dont_initialize();
    }

    void load_instruction(Instruction inst) { //Testbench calls this to push inputs in
        m_instruction_mem.push(inst);
    }

private:

```

```

size_t m_num_stages;
std::vector<std::optional<Instruction>> m_stages;
std::queue<Instruction> m_instruction_mem;
bool m_downstream_stalled = false;

void tick() {
    std::cout << "\n—— [Cycle " << sc_core::sc_time_stamp() << "]" << "\n";

```

```

    // 1. OUTBOUND PATH

```

```

    if (m_stages.back().has_value()) {
        if (!m_downstream_stalled) {
            tlm::tlm_phase phase = tlm::BEGIN_REQ;
            sc_core::sc_time delay = sc_core::SC_ZERO_TIME;

```

```

            // Peek at the data without removing it yet

```

```

            //std::optional<Instruction> tx = m_stages.back();

```

```

            //tlm::tlm_sync_enum status = init_socket->nb_transport_fw(tx, phase, delay);

```

```

            // improved below - pass a direct reference to the object inside the vector-ZERO allocation, ZERO

```

copying

```

            std::optional<Instruction>& tx_ref = m_stages.back();
            tlm::tlm_sync_enum status = init_socket->nb_transport_fw(tx_ref, phase, delay);

```

```

            if (status == tlm::TLM_UPDATED && phase == custom_phase::BUFFER_FULL) {
                std::cout << " [Pipeline] FIFO full warning! Halting future dispatches.\n";
                m_downstream_stalled = true;
                m_stages.back() = std::nullopt; // Consumed! Stage 4 is now EMPTY.
            } else if (status == tlm::TLM_COMPLETED) {
                std::cout << " [Pipeline] Retiring " << m_stages.back()->op << "\n";
                m_stages.back() = std::nullopt; // Consumed! Stage 4 is now EMPTY.
            }

```

```

        } else {
            std::cout << " [Pipeline] Outbound blocked. Holding " << m_stages.back()->op
                << " in Stage " << m_num_stages - 1 << "\n";
        }
    }

```

```

    // 2. INTERNAL SHIFT & FETCH PATH

```

```

// ONLY freeze the internal shift if the final stage is physically occupied.
if (m_stages.back().has_value()) {
    std::cout << "    [Pipeline] STALLED. Stage " << m_num_stages - 1 << " cannot clear.\n";
} else {
    // Stage 4 is empty (either it was already empty, or it just successfully retired).
    // It is safe to shift earlier stages forward!
    for (size_t i = m_num_stages - 1; i > 0; --i) {
        if (m_stages[i - 1].has_value() && !m_stages[i].has_value()) {
            m_stages[i] = std::move(m_stages[i - 1]);
            m_stages[i - 1] = std::nullopt;
        }
    }

    // Fetch new instruction into Stage 0
    if (!m_stages[0].has_value() && !m_instruction_mem.empty()) {
        m_stages[0] = m_instruction_mem.front();
        m_instruction_mem.pop();
        std::cout << "    [Pipeline] Fetched " << m_stages[0]->op << " into Stage 0\n";
    }
}

print_pipeline_state();
}

tlm::tlm_sync_enum nb_transport_bw(std::optional<Instruction>& tx, tlm::tlm_phase& phase, sc_core::sc_time&
delay) {
    if (phase == custom_phase::BUFFER_FREE) {
        std::cout << "    [Pipeline] Received BUFFER_FREE notification. Resuming operations next cycle.\n";
        m_downstream_stalled = false;
        return tlm::TLM_COMPLETED;
    }
    return tlm::TLM_ACCEPTED;
}

void print_pipeline_state() {
    std::cout << "    [Stage Map] ";
    for (size_t i = 0; i < m_num_stages; ++i) {

```

```

        std::cout << "S" << i << ":[ " << (m_stages[i].has_value() ? m_stages[i]->op : "---") << "]" << " ";
    }
    std::cout << "\n";
}
};

// =====
// MODULE 2: PROGRAMMABLE SIZE FIFO CHANNEL
// =====
class ProgrammableFifo : public sc_core::sc_module {
public:
    tlm_utils::simple_target_socket<ProgrammableFifo, 32, CA_Traits> target_socket;
    tlm_utils::simple_initiator_socket<ProgrammableFifo, 32, CA_Traits> init_socket;
    sc_core::sc_in<bool> clk;

    SC_HAS_PROCESS(ProgrammableFifo);

    ProgrammableFifo(sc_core::sc_module_name name, size_t fifo_size = 4)
        : sc_core::sc_module(name), target_socket("target_socket"), init_socket("init_socket"),
m_max_size(fifo_size)
    {
        target_socket.register_nb_transport_fw(this, &ProgrammableFifo::nb_transport_fw);
        init_socket.register_nb_transport_bw(this, &ProgrammableFifo::nb_transport_bw);

        SC_METHOD(drain_cycle);
        sensitive << clk.pos();
        dont_initialize();
    }

private:
    size_t m_max_size;
    std::queue<Instruction> m_storage;
    bool m_upstream_notified_full = false;
    bool m_sink_stalled = false;

    tlm::tlm_sync_enum nb_transport_fw(std::optional<Instruction>& tx, tlm::tlm_phase& phase, sc_core::sc_time&
delay) {

```

```

    if (phase == tlm::BEGIN_REQ && tx.has_value()) {
        if (m_storage.size() >= m_max_size) {
            return tlm::TLM_REJECTED;
        }

        m_storage.push(*tx);
        std::cout << "                [FIFO] Accepted: " << tx->op << " (Slots filled: " << m_storage.size() <<
"/" << m_max_size << ")\n";

        if (m_storage.size() >= m_max_size) {
            phase = custom_phase::BUFFER_FULL;
            m_upstream_notified_full = true;
            return tlm::TLM_UPDATED;
        }
        return tlm::TLM_COMPLETED;
    }
    return tlm::TLM_ACCEPTED;
}

void drain_cycle() {
    if (m_storage.empty()) return;

    if (m_sink_stalled) {
        std::cout << "                [FIFO] Blocked. Downstream Execution Unit is busy.\n";
        return;
    }

    // Packaged item cleanly back inside an optional wrapper for delivery
    std::optional<Instruction> tx_out = m_storage.front();
    //just changing this to tx_ref (for efficiency/perf reason) do NOT work like the case in module #1 above
    //as the storage here is queue (above is vector) and
    //m_storage.pop() destroys the element at the front and frees or moves its memory
    //std:vector needs to be used again but this case extra logic would then be needed
    tlm::tlm_phase phase = tlm::BEGIN_REQ;
    sc_core::sc_time delay = sc_core::SC_ZERO_TIME;

    std::cout << "                [FIFO] Attempting to dispatch " << tx_out->op << " to execution sink\n";

```

```

    tlm::tlm_sync_enum status = init_socket->nb_transport_fw(tx_out, phase, delay);

    if (status == tlm::TLM_UPDATED && phase == custom_phase::BUFFER_FULL) {
        std::cout << "                [FIFO] Destination rejected/stalled. Halting dispatch.\n";
        m_sink_stalled = true;
    } else {
        m_storage.pop();

        if (m_upstream_notified_full && m_storage.size() < m_max_size) {
            m_upstream_notified_full = false;
            std::optional<Instruction> empty_signal = std::nullopt;
            tlm::tlm_phase release_phase = custom_phase::BUFFER_FREE;
            target_socket->nb_transport_bw(empty_signal, release_phase, delay);
        }
    }
}

    tlm::tlm_sync_enum nb_transport_bw(std::optional<Instruction>& tx, tlm::tlm_phase& phase, sc_core::sc_time&
delay) {
    if (phase == custom_phase::BUFFER_FREE) {
        std::cout << "                [FIFO] Received execution unit clearance. Resuming next cycle.\n";
        m_sink_stalled = false;
        return tlm::TLM_COMPLETED;
    }
    return tlm::TLM_ACCEPTED;
}
};

// =====
// MODULE 3: DOWNSTREAM EXECUTION SINK
// =====
class ExecutionSink : public sc_core::sc_module {
public:
    tlm_utils::simple_target_socket<ExecutionSink, 32, CA_Traits> target_socket;
    sc_core::sc_in<bool> clk;

    SC_HAS_PROCESS(ExecutionSink);

```

```

ExecutionSink(sc_core::sc_module_name name) : sc_core::sc_module(name), target_socket("target_socket") {
    target_socket.register_nb_transport_fw(this, &ExecutionSink::nb_transport_fw);
    SC_METHOD(execute_cycle);
    sensitive << clk.pos();
    dont_initialize();
}

private:
    std::optional<Instruction> m_active_job = std::nullopt;
    bool m_sink_busy = false;

    tlm::tlm_sync_enum nb_transport_fw(std::optional<Instruction>& tx, tlm::tlm_phase& phase, sc_core::sc_time&
delay) {
        if (phase == tlm::BEGIN_REQ && tx.has_value()) {
            if (m_sink_busy) {
                phase = custom_phase::BUFFER_FULL;
                return tlm::TLM_UPDATED;
            }
            m_active_job = std::move(*tx);
            m_sink_busy = true;
            return tlm::TLM_COMPLETED;
        }
        return tlm::TLM_ACCEPTED; // Catch-all safety net for unhandled protocol phases
    }

    void execute_cycle() {
        if (!m_active_job.has_value()) return;

        if (m_active_job->delay_cycles > 0) {
            std::cout << "                                [SINK] Processing " << m_active_job->op << ". Cycles Remaining:
" << m_active_job->delay_cycles << "\n";
            m_active_job->delay_cycles--;
        } else {
            std::cout << "                                [SINK] SUCCESS: Retired instruction " << m_active_job->op <<
"\n";
            m_active_job = std::nullopt;

```

```

        m_sink_busy = false;

        // Notify upstream FIFO that we can accept work again
        std::optional<Instruction> empty = std::nullopt;
        tlm::tlm_phase free_phase = custom_phase::BUFFER_FREE;
        sc_core::sc_time delay = sc_core::SC_ZERO_TIME;
        target_socket->nb_transport_bw(empty, free_phase, delay);
    }
};

// =====
// TESTBENCH TOP
// =====
int sc_main(int argc, char* argv[]) {
    sc_core::sc_clock clk("system_clock", 10, sc_core::SC_NS);

    ProgrammablePipeline pipeline("pipeline", 5);
    ProgrammableFifo internal_fifo("internal_fifo", 2);
    ExecutionSink execution_unit("execution_unit");

    pipeline.init_socket.bind(internal_fifo.target_socket);
    internal_fifo.init_socket.bind(execution_unit.target_socket);

    pipeline.clk(clk);
    internal_fifo.clk(clk);
    execution_unit.clk(clk);

    pipeline.load_instruction({0x100, "ADD", 0});
    pipeline.load_instruction({0x104, "SUB", 0});
    pipeline.load_instruction({0x108, "MUL", 3});
    pipeline.load_instruction({0x10C, "AND", 0});
    pipeline.load_instruction({0x110, "XOR", 0});
    pipeline.load_instruction({0x114, "NOR", 0});

    std::cout << "Beginning Multi-Module Backpressure Simulation...\n";
    sc_core::sc_start(160, sc_core::SC_NS);
}

```

```
std::cout << "Simulation execution completed gracefully.\n";

return 0;
}
```

Beginning Multi-Module Backpressure Simulation...

```
— [Cycle 10 ns] —
[Pipeline] Fetched ADD into Stage 0
[Stage Map] S0:[ADD] S1:[---] S2:[---] S3:[---] S4:[---]

— [Cycle 20 ns] —
[Pipeline] Fetched SUB into Stage 0
[Stage Map] S0:[SUB] S1:[ADD] S2:[---] S3:[---] S4:[---]

— [Cycle 30 ns] —
[Pipeline] Fetched MUL into Stage 0
[Stage Map] S0:[MUL] S1:[SUB] S2:[ADD] S3:[---] S4:[---]

— [Cycle 40 ns] —
[Pipeline] Fetched AND into Stage 0
[Stage Map] S0:[AND] S1:[MUL] S2:[SUB] S3:[ADD] S4:[---]

— [Cycle 50 ns] —
[Pipeline] Fetched XOR into Stage 0
[Stage Map] S0:[XOR] S1:[AND] S2:[MUL] S3:[SUB] S4:[ADD]

— [Cycle 60 ns] —
[FIFO] Accepted: ADD (Slots filled: 1/2)
[Pipeline] Retiring ADD
[FIFO] Attempting to dispatch ADD to execution sink
[SINK] SUCCESS: Retired instruction ADD
[Pipeline] Fetched NOR into Stage 0
[Stage Map] S0:[NOR] S1:[XOR] S2:[AND] S3:[MUL] S4:[SUB]

— [Cycle 70 ns] —
[FIFO] Accepted: SUB (Slots filled: 1/2)
```

[Pipeline] Retiring SUB
[FIFO] Attempting to dispatch SUB to execution sink
[SINK] SUCCESS: Retired instruction SUB
[Stage Map] S0:[---] S1:[NOR] S2:[XOR] S3:[AND] S4:[MUL]

— [Cycle 80 ns] —
[FIFO] Accepted: MUL (Slots filled: 1/2)
[Pipeline] Retiring MUL
[FIFO] Attempting to dispatch MUL to execution sink
[SINK] Processing MUL. Cycles Remaining: 3
[Stage Map] S0:[---] S1:[---] S2:[NOR] S3:[XOR] S4:[AND]

— [Cycle 90 ns] —
[FIFO] Accepted: AND (Slots filled: 1/2)
[Pipeline] Retiring AND
[FIFO] Attempting to dispatch AND to execution sink
[FIFO] Destination rejected/stalled. Halting dispatch.
[SINK] Processing MUL. Cycles Remaining: 2
[Stage Map] S0:[---] S1:[---] S2:[---] S3:[NOR] S4:[XOR]

— [Cycle 100 ns] —
[FIFO] Accepted: XOR (Slots filled: 2/2)
[Pipeline] FIFO full warning! Halting future dispatches.
[FIFO] Blocked. Downstream Execution Unit is busy.
[SINK] Processing MUL. Cycles Remaining: 1
[Stage Map] S0:[---] S1:[---] S2:[---] S3:[---] S4:[NOR]

— [Cycle 110 ns] —
[Pipeline] Outbound blocked. Holding NOR in Stage 4
[Pipeline] STALLED. Stage 4 cannot clear.
[FIFO] Blocked. Downstream Execution Unit is busy. {note: maybe this log should also show this here:
"Slots filled: 2/2" }
[SINK] SUCCESS: Retired instruction MUL
[FIFO] Received execution unit clearance. Resuming next cycle.
[Stage Map] S0:[---] S1:[---] S2:[---] S3:[---] S4:[NOR]

— [Cycle 120 ns] —

```

[Pipeline] Outbound blocked. Holding NOR in Stage 4
[Pipeline] STALLED. Stage 4 cannot clear.
[FIFO] Attempting to dispatch AND to execution sink
[Pipeline] Received BUFFER_FREE notification. Resuming operations next cycle.
[SINK] SUCCESS: Retired instruction AND

{note: maybe this log should also show this here: "[FIFO] Slots filled: 1/2" }
```

```

[Stage Map] S0:[---] S1:[---] S2:[---] S3:[---] S4:[NOR]
```

```

— [Cycle 130 ns] —
```

```

[FIFO] Accepted: NOR (Slots filled: 2/2)
```

```

[Pipeline] FIFO full warning! Halting future dispatches.
```

```

[FIFO] Attempting to dispatch XOR to execution sink
```

```

[Pipeline] Received BUFFER_FREE notification. Resuming operations next cycle.
```

```

[SINK] SUCCESS: Retired instruction XOR
```

```

[Stage Map] S0:[---] S1:[---] S2:[---] S3:[---] S4:[---]
```

```

— [Cycle 140 ns] —
```

```

[FIFO] Attempting to dispatch NOR to execution sink
```

```

[SINK] SUCCESS: Retired instruction NOR
```

```

[Stage Map] S0:[---] S1:[---] S2:[---] S3:[---] S4:[---]
```

```

— [Cycle 150 ns] —
```

```

[Stage Map] S0:[---] S1:[---] S2:[---] S3:[---] S4:[---]
```

```

— [Cycle 160 ns] —
```

```

[Stage Map] S0:[---] S1:[---] S2:[---] S3:[---] S4:[---]
```

```

Simulation execution completed gracefully.
```

TO-DOs (through other examples):

- using tlm_generic_payload acquire() and release() as well as Payload Event Queues
- NOT using clk but still Approximately Timed, to help simulation speed
- capture statistics, like how often a certain FIFO is full (overall vs ONLY count after ramp-up and before ramp-down [like window starting when the 1st item pushed and ending when the last item got popped in the workload]) => Done (see next example)